arXiv:2407.00320v1 [cs.CL] 29 Jun 2024

LiteSearch: Efficacious Tree Search for LLM

Ante Wang[1], **Linfeng Song**[2*], **Ye Tian**[2], **Baolin Peng**[2], **Dian Yu**[2], **Haitao Mi**[2], **Jinsong Su**[1*] and **Dong Yu**[2]
[1]School of Informatics, Xiamen University, China
[2]Tencent AI Lab, Bellevue, WA
wangante@stu.xmu.edu.cn, lfsong@global.tencent.com, jssu@xmu.edu.cn

Abstract

Recent research suggests that tree search algorithms (e.g. Monte Carlo Tree Search) can dramatically boost LLM performance on complex mathematical reasoning tasks. However, they often require more than 10 times the computational resources of greedy decoding due to wasteful search strategies, making them difficult to be deployed in practical applications. This study introduces a novel guided tree search algorithm with dynamic node selection and node-level exploration budget (maximum number of children) calculation to tackle this issue. By considering the search progress towards the final answer (history) and the guidance from a value network (future) trained without any step-wise annotations, our algorithm iteratively selects the most promising tree node before expanding it within the boundaries of the allocated computational budget. Experiments conducted on the GSM8K and TabMWP datasets demonstrate that our approach not only offers competitive performance but also enjoys significantly lower computational costs compared to baseline methods.

1 Introduction

Mathematical reasoning tasks (Amini et al., 2019; Cobbe et al., 2021; Hendrycks et al., 2021; Lu et al., 2022) have long been acknowledged as challenging. These tasks require transforming a question into a sequence of reasoning steps, which are subsequently executed to derive the correct answer. Recently, large language models (LLMs, Achiam et al. 2023; Touvron et al. 2023; Jiang et al. 2024) have demonstrated remarkable potential in addressing them. A pivotal approach is the employment of Chain-of-Thought (CoT) prompting (Wei et al., 2022; Kojima et al., 2022), which prompts LLMs to break down a question solution into a sequence of reasoning steps before reaching an answer.

*Corresponding Authors

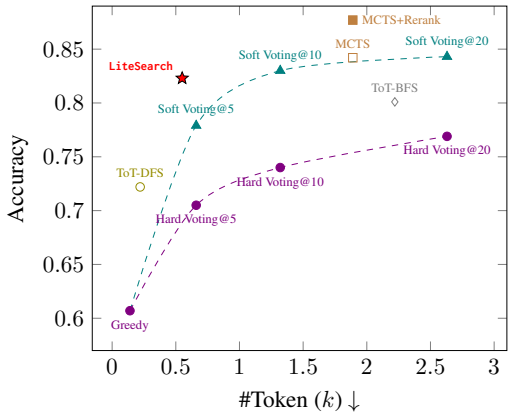

Figure 1: Comparison among ours and typical systems on GSM8K, where *DFS*, *BFS*, *MCTS* (Tian et al., 2024)[1] and LiteSearch are only guided by the same value network, and *MCTS+Rerank* additionally reranks all searched complete trajectories with value network. *Hard Voting* is self-consistency (Wang et al., 2022) and *Soft Voting* additionally use value network to weight each trajectory. We measure the number of generated tokens (*#Tokens (k)*) by the LLM as computation costs.

Despite their impressive capabilities, LLMs still face challenges when tackling problems with increasing reasoning steps due to the nature of autoregressive decoding. This can be analogous to the "System 1" mode of thought in psychology (Daniel, 2017), which is characterized by fast, intuitive, but error-prone thinking. Much of recent work has focused on enhancing the "System 1" capability of LLMs by prompt-engineering, such as hierarchical prompting (Suzgun and Kalai, 2024; Zeng et al., 2023) and automatic prompt refine (Madaan et al., 2024; Yang et al., 2024; Zhou et al., 2024). On the other hand, growing research attention is being paid to promote the "System 2" mode of thought (Daniel, 2017) for LLMs, which is characterized by deliberative thinking steps with back-and-forth refinements. These are the key features for solving

[1]The MCTS here does not utilize the guidance from PRM or ORM for fair comparison and efficiency.

complex math reasoning tasks. Particularly, prior efforts have studied enhancing LLMs both at inference time and through self-improvement using tree search algorithms (e.g., DFS and BFS, Yao et al. 2024) and Monte Carlo Tree Search (MCTS, Feng et al. 2023; Tian et al. 2024; Zhang et al. 2024; Wang et al. 2024b).

However, these approaches often necessitate the creation of expert-designed utility functions (Tian et al., 2024; Ma et al., 2023; Kang et al., 2024), making them difficult to be adapted to new scenarios. Moreover, they are computationally intensive, especially when tackling problems that require numerous logical steps (Xie et al., 2024). This is because these methods ineffectively manage the expansion budget (the number of nodes to expand) throughout the search process. As a typical example, BFS adopts a constant budget size throughout the search process, overlooking the fact that some tree nodes do not require much expansion. Some MCTS approaches (Tian et al., 2024) take adaptive budget based on the importance of each node, but they still require a large number of simulations or rollouts for accurate statistics to make decisions, and they overlook other important information, such as the depth (progress) of each node. As the result, there is a pressing need to develop more efficient and adaptable methods for enhancing LLMs' "System 2" reasoning capabilities to effectively handle complex reasoning tasks.

In this study, we introduce a guided tree search algorithm with dynamic node selection and node-level exploration budget calculation, aiming to maintain the performance at a moderate cost. Concretely, we employ the value score as guidance to *select* the most promising node for the next action and *expand* it within a dynamically computed budget size, navigating exploration-exploitation balance for guided tree search. We continue iterating operations of selection and expansion until the resulting trajectory either meets the expected quality score or surpasses the maximum number of iterations. Notably, the computational budget for each node is inversely correlated to its value score. This is inspired by the observation that nodes with higher value scores are more likely to yield the correct solution upon expansion, hence we allocate fewer computational resources to them to prevent unnecessary computation and vice versa. This not only promotes efficient exploitation, facilitating a faster convergence to the final answer, but also guarantees sufficient exploration to cover enough state space for maintaining performance.

We conduct experiments on popular GSM8K (Cobbe et al., 2021) and TabMWP (Lu et al., 2022). Results show that our methods offer competitive performance but significantly less computation costs (saving around 5×) compared to other baselines. Detailed analyses confirm the usefulness of each component and provide more practical options for various settings. Additionally, we also identify the limitations of this research line and suggest possible ways to tackle them.

2 Related Work

Thanks to the robust capabilities of LLMs, significant advancements have been made in mathematical reasoning tasks, surpassing traditional approaches that rely on semantic parsing (Matsuzaki et al., 2017; Hopkins et al., 2017) or Abstract Syntax Tree (AST) decoding (Li et al., 2019; Qin et al., 2021; Wu et al., 2021).

Some studies improved the reasoning capabilities of LLMs through further training. These efforts involved either manually annotating or automatically generating feasible and challenging problems to fine-tune the LLMs (Luo et al., 2023; Yu et al., 2023; Liu and Yao, 2024; Toshniwal et al., 2024), as well as devising sophisticated techniques, such as reinforcement learning, for efficient training (Luo et al., 2023; Wang et al., 2023; Lightman et al., 2023; Chen et al., 2024).

Another line of research focuses on inference-time improvement. Except for the popular self-consistency (Wang et al., 2022), most of these studies treat this task as a tree search problem and investigate various searching algorithms. Yao et al. (2024) were the first to introduce Tree-of-Thought (ToT), incorporating Depth-First Search (DFS) and Breath-First Search (BFS) to address reasoning problems. Khalifa et al. (2023); Zhu et al. (2024); Xie et al. (2024) applied step-wise Beam Search to math problems, which operates similarly to BFS under certain parameter conditions. To guide the search process, these studies above either directly prompt the LLMs to evaluate the quality of each step (Yao et al., 2024; Xie et al., 2024), or train a verifier on corresponding datasets to achieve better performance (Khalifa et al., 2023; Zhu et al., 2024).

Later research delved into other sophisticated search algorithms, such as Monte Carlo Tree Search (MCTS, Tian et al. 2024; Zhang et al. 2024; Wang et al. 2024b), A* (Ma et al., 2023), and

Levin Tree Search (Kang et al., 2024). Nonetheless, these approaches necessitate more robust verifiers to steer the search procedure. Concretely, Tian et al. (2024) utilize a blend of the value function, Process-supervised Reward Model (PRM), and Outcome-supervised Reward Model (ORM). Ma et al. (2023) and Kang et al. (2024) train their PRM models on PRM800K (Lightman et al., 2023), which offers manual annotations for $800k$ reasoning steps of problems from MATH (Hendrycks et al., 2021).

This study also follows the same research line, yet it concentrates on developing an efficient algorithm to decrease computation costs while maintaining performance. Besides, we employ a naive but more practical value network as the verifier, which is trained solely with the final answer labels as distant supervision.

3 LiteSearch

Algorithm 1 LiteSearch

Require: question q, maximum iterations N, threshold ε, policy π, value network v

```
Initialize tree T with q as the root
Set i ← 0, ŷ ← null
while i < N do
    Select node s' from T using Eq. 2
    Expand s' to obtain its child nodes C using Eq. 3
    for c ∈ C do
        S ← return_path(T, c)
        if is_terminal(S) and v(S) > ε then
            Set ŷ ← S
            break
        end if
    end for
    i ← i + 1
end while
```

Ensure: $\hat{y}$

3.1 Guided Tree Search Algorithm

Taking each math reasoning question q as a tree search problem, we initialize the root of the search tree with question q, while the other tree nodes represent reasoning steps (e.g., s_i) generated by an LLM (denoted as policy π). Concretely, we treat an (incomplete) trajectory $q, s_1, ..., s_i$ as the state $\mathcal{S}_i$.[2] Then, a next step can be sampled from the LLM which consumes $\mathcal{S}_i$:

$$s_{i+1} \sim \texttt{LLM}(\mathcal{D}, \mathcal{S}_i), \tag{1}$$

where $\mathcal{D}$ is the in-context demonstrations made of question-solution pairs.

As shown in Alg. 1 and Fig. 2, our algorithm mainly comprises an iterative process of *Selection* (§3.1.1) and *Expansion* (§3.1.2) operations. For each loop, we first *select* the most promising node, and then *expand* it within the constraints of the computational budget. Both operations are guided by a value network v (§3.2). The algorithm terminates when the generated answers meet the expected value threshold ε or the number of iterations reaches the limit N.

3.1.1 Selection

We mainly select the tree node with the highest value for expansion. Besides, we introduce a *progress term*, denoted as $p(\mathcal{S})$, which quantifies the advancement of a state $\mathcal{S}$ towards the goal within the search trajectory. By incorporating this term, we prioritize the exploration of nodes that are expected to lead more rapidly to the final answer.

$$s' = \max_{s_i}(v(\mathcal{S}_i) + \lambda p(\mathcal{S}_i)), \tag{2}$$

where s' denotes the selected node, and λ is introduced to regulate the impact of the progress term.

It is non-trivial to estimate the progress of a state, thus we introduce an empirical approach based on the trajectory of greedy decoding. Specifically, we compute the progress by comparing the number of tokens or steps from a given state to those of the corresponding greedy decoding. For example, when using step number as the metric, a state with d steps has progress of $d/\hat{d}$, where $\hat{d}$ denotes the total number of steps in the greedy decoded trajectory.

3.1.2 Expansion

During the expansion phase, we aim to balance exploitation and exploration by effectively managing the computation budget allocated to the selected node. Intuitively, an appropriate budget size can promote efficient exploitation, facilitating a faster convergence to the final answer, while also guaranteeing sufficient exploration to cover enough state space for reducing uncertainty. In line with this spirit, we further explore two strategies preferring either exploitation or exploration: *Incremental Expansion* and *Batch Expansion*.

[2] Specially, we define $\mathcal{S}_0 = q$.

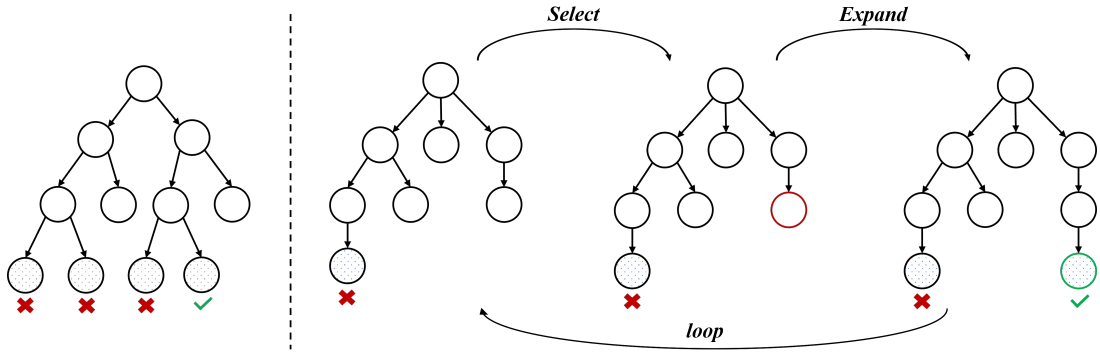

Figure 2: Framework of our guided tree search (Right), which is an iteration of two major steps: *select* and *expand*. Compared with a previous method (BFS, Left), our method dynamically selects the most promising node and expands it within an offered budget, thus reaching the answer without visiting unnecessary nodes.

Budget Computaton We define the allocated budget for a node $\mathcal{S}$ as the maximum number of its children, denoted as b, which primarily depends on the value $v(\mathcal{S})$ and depth d of that node.

$$b = \min\left(\lceil \frac{\log(1-\epsilon)}{d\log(1-v(\mathcal{S}))} \rceil, B\right), \quad (3)$$

where B denotes the upper bound of the budget and ϵ is the expected accuracy, thus a larger ϵ (e.g., 0.95) encourages more conservative searching. Besides, we employ the $1/d$ term, which fosters exploration at the start of searching but encourages exploitation with d increasing to avoid search space explosion.

As the value scores of the preceding search steps usually suffer a larger variance due to inefficient learning of delayed and sparse rewards (Sutton and Barto, 2018), confidence estimation of them is relatively not accurate enough.[3] This inevitably influences the computation of suitable budget sizes. Therefore, we propose to further calibrate value scores using the values of corresponding trajectory from greedy decoding (denoted as $\hat{v}$), especially for the first few steps.

$$v'(S) = \frac{v(\mathcal{S}) + \hat{v}/d}{1 + 1/d}, \quad (4)$$

where $v'(S)$ represents the enhanced value calibrated by $\hat{v}$ after normalization. We add $1/d$ term to mainly help the first several steps.

Expansion Strategies We propose two expansion strategies that prioritize efficiency and performance, respectively.

- **Incremental Expansion**: This approach incrementally expands one child node after another. If the budget allows, the same node can be reselected until the budget is fully utilized. This method tends to conserve computational resources by carefully managing the budget.
- **Batch Expansion**: In contrast, this strategy consumes the entire budget allocated to a node during each iteration, resulting in the generation of multiple child nodes simultaneously. This method broadens the search space for subsequent iterations, potentially leading to the identification of superior nodes and enhancing overall performance.

3.2 Value Network

The value network $v(\mathcal{S})$ seeks to approximate the expected cumulative reward starting from state $\mathcal{S}$ and following a policy π thereafter. This can be represented as $v(\mathcal{S}) = \mathbb{E}_\pi\left[R_t \mid \mathcal{S}_t = \mathcal{S}\right]$, where R_t is the discounted return starting from state $\mathcal{S}_t$.

Particularly, given a question q and its correct answer y from an expert demonstration dataset. Each trajectory with reasoning steps (e.g., s_i) and final predicted answer $\hat{y}$ is firstly sampled from the LLM (policy π):

$$s_1, ..., s_n, \hat{y} \sim \texttt{LLM}(\mathcal{D}, q). \quad (5)$$

Then, we only take the answer correctness as distant supervision for each reasoning step to train the value network via Mean Squared Error (MSE) loss:

$$\mathcal{L} = (v(\mathcal{S}_i) - \mathbb{I}[y = \hat{y}])^2, \quad (6)$$

where $\mathbb{I}$ denotes an indicator function.

In this work, regardless of the policy used, we simply take Llama3-8B[4] with a regressive head as our value network. This regressive head is a

[3] This can also be observed in Fig. 3.

[4] https://ai.meta.com/blog/meta-llama-3/

randomly initialized linear layer, which consumes the hidden state of the last input token and returns a scalar within $[0, 1]$:

$$v(\mathcal{S}_i) = \texttt{Head}(\texttt{Llama3}(P, \mathcal{S}_i)[-1]), \quad (7)$$

where P is an instruction to help learning. An example is shown in the Fig. 9.

4 Experiment

4.1 Setup

Dataset We conduct experiments on two popular mathematical reasoning datasets:

- **GSM8K** (Cobbe et al., 2021): This dataset comprises 7,473 training and 1,319 testing grade school math word problems that take $2 \sim 8$ steps to solve. Solutions primarily involve performing a sequence of elementary calculations using basic arithmetic operations.
- **TabMWP** (Lu et al., 2022): This dataset features 38,431 tabular math word problems, presented in either free-text or multiple-choice formats. We focus on the more general free-text category, consisting of 17,315 training questions and 1,000 randomly sampled test questions for evaluation. In contrast to GSM8K, reasoning in TabMWP is based on the provided tables. Additionally, it spans a wider range of domains and supports various answer types.

Models and Hyperparameters We employ Mixtral-8×7B (Jiang et al., 2024) or Llama3-8B as the policy model and train Llama3-8B as the value network. For the policy models, we adhere to the standard approach of utilizing 8 / 4 shots in-context learning for GSM8K / TabMWP, with a temperature of 0.6. By default, we set $N, B, \lambda, \varepsilon, \epsilon$ as 100, 10, 0, 0.8, and 0.9, respectively, and investigate other combinations in our analyses. For the value networks, we sample 8 trajectories per training instance, also with a temperature of 0.6. Then, we train the models for 1 epoch across both datasets, employing the AdamW optimizer (Loshchilov and Hutter, 2017) with a learning rate of 5e-6 and a linear learning rate scheduler. Besides, we allocate 5% of the training instances as a development set to select the optimal checkpoints as value networks.

Evaluation Metrics We adopt answer *Accuracy* and the number of generated tokens (*Tokens (k)*) as evaluation metrics for performance and cost, respectively. It should be noted that we do not take into account the cost of executing value networks. This is because a value network only performs the regression task, which incurs significantly lower costs compared to the primary generation task. Besides, it also can be deployed in parallel in practice.

Baselines We consider the following baselines:

- **Greedy Decoding**: It intuitively selects the most probable next token at each decoding step.
- **Hard Voting@K (SC,** Wang et al. 2022**)**: Known as self-consistency, which ensembles the answers from multiple sampled solutions as the final answer using majority voting. We sample $K = \{5, 10, 20\}$ times with a temperature of 0.6.
- **ToT-DFS** (Yao et al., 2024): We implement it by capitalizing on guidance from our trained value network. Specifically, we prune a node if its value score falls below a threshold of 0.5 and limit the maximum number of children to 5 to prevent infinite loops.
- **ToT-BFS / BeamSearch** (Khalifa et al., 2023; Yao et al., 2024; Xie et al., 2024; Zhu et al., 2024): These two methods work similarly for this task. Again leveraging our value networks, we ask each node to expand 5 children and only keep 5 nodes with the highest value scores at each depth to avoid search space explosion.
- **Soft Voting@K**: It is an enhancement over hard voting by utilizing our value networks. It softly ensembles the answers of different paths by taking their value scores as weights.

4.2 Development Experiments

As depicted in Fig. 3, we analyze the alignment between value scores and final correctness. We employ the widely-used Brier score as our evaluation metric, which is the MSE in effect, indicating that lower scores are preferred.

Generally, as the number of steps increases, we notice a descending trend. This observation suggests a better correlation between the value scores and the final correctness in subsequent steps. Remarkably, the Brier scores span from 0.21 to 0.14, marking an improvement of merely 0.07. This echoes the conclusion of previous research (Tian et al., 2024) and paves the way for our method by taking value scores as estimated confidence of final correctness.

		GSM8K		TabMWP	
		Accuracy ↑	Tokens (k) ↓	Accuracy ↑	Tokens (k) ↓
Mixtral-8×7B	Greedy Decoding	.607	0.14	.762	0.07
	Hard Voting@5	.705	0.66	.761	0.37
	Hard Voting@10	.740	1.32	.782	0.73
	Hard Voting@20	.769	2.63	.796	1.46
	ToT-DFS	.722	**0.22**	.822	**0.16**
	ToT-BFS	.801	2.22	.861	1.45
	Soft Voting@5	.779	0.66	.811	0.37
	Soft Voting@10	.830	1.32	.832	0.73
	Soft Voting@20	**.843**	2.63	.847	1.46
	Ours (Incremental)	.797	0.41	**.863**	0.22
	Ours (Batch)	.823	0.55	.854	0.29
Llama3-8B	Greedy Decoding	.485	0.18	.659	0.08
	Hard Voting@5	.572	0.57	.680	0.42
	Hard Voting@20	.667	2.38	.698	1.68
	ToT-DFS	.676	**0.24**	.704	**0.19**
	ToT-BFS	.756	1.89	.787	1.35
	Soft Voting@5	.689	0.57	.747	0.42
	Soft Voting@20	**.770**	2.38	**.796**	1.68
	Ours (Incremental)	.731	0.46	.779	0.27
	Ours (Batch)	.757	0.59	.776	0.35

Table 1: Main test results. For methods guided by our value networks, we emphasize the best results in **bold** and the second / third-best results with underlining.

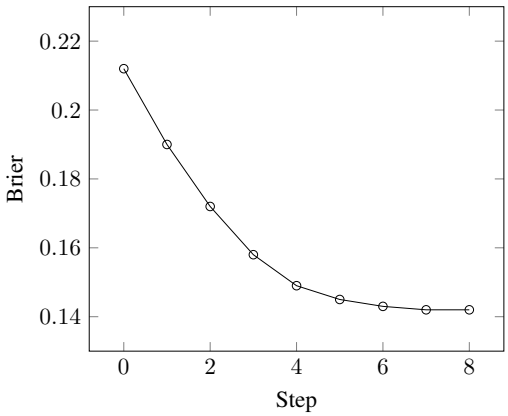

Figure 3: Reliability of confidence estimation using value scores at different reasoning steps.

4.3 Main Results

Table 1 shows the main test results on GSM8K and TabMWP. We observe the following conclusions:

Value Guidance Boosts Model Performance In line with prior research (Wang et al., 2022), Hard Voting significantly improves Accuracy. However, its costs also proportionately increase with the growing of sampling size K. With the guidance of our value networks, both Soft Voting and tree search algorithms can further enhance Accuracy without incurring additional costs. Besides, *Soft Voting@5* consistently surpasses *Hard Voting@20*, substantiating the effectiveness of verification as previously discussed in (Cobbe et al., 2021).

Current Tree Search Algorithms Neglect the Performance-Cost Tradeoff Previous methods, *ToT-DFS* and *ToT-BFS*, prefer different evaluation metrics. Among the value-guided approaches, *ToT-DFS* consistently has the lowest cost but achieves suboptimal performance. This is because *ToT-DFS* focuses mainly on pruning bad nodes and lacks the flexibility to select better nodes for further improvement. In contrast, *ToT-BFS* tackles this shortcoming of *ToT-DFS* by maintaining a branch of nodes with the highest values, thereby resulting in better performance. However, it also unnecessarily visits lots of nodes during the search, leading to significantly higher costs.

Dynamic Selection and Expansion Maintain Performance and Decrease Cost By fully utilizing the guidance from value networks, our methods achieve the best tradeoff between performance and cost. Our approach falls within the cost range of *ToT-DFS* and *Soft Voting@5*, yet yields significantly better performance. For the two expansion strategies, *Incremental* saves nearly 20% of costs of *Batch* and performs even better on TabMWP. However, *Incremental* performs noticeably worse than *Batch* on Accuracy on GSM8K, with a 2.6-point lower score. This is due to *Batch* providing a better

	Accuracy ↑	Tokens (k) ↓
Incremental	**.797**	0.41
⇒ static budget	.779	0.67
w/o depth penalty	.780	0.43
w/o greedy value	.783	**0.40**
Batch	**.823**	**0.55**
⇒ static budget	.802	1.79
w/o depth penalty	.815	0.79
w/o greedy value	.806	0.62

Table 2: Ablation study on dynamic budgeting.

comparison among nodes for selection by expanding more nodes each time. It is worth noting that both of our methods often cannot outperform *Soft Voting@20* on Accuracy. We will provide detailed analyses in §4.5.

4.4 Ablation Study and Analyses

Dynamic Budgeting Helps Both Performance and Cost-Efficiency We first study the effectiveness of the dynamic budget size b, which is decided by Eq. 3. The following variants are considered: (1) ⇒ *static budget*: We directly set b as B, resulting in each node being expanded with a fixed budget size; (2) *w/o depth penalty*: We remove the $1/d$ term from Eq. 3, which previously penalized b as the depth d increased; (3) *w/o greedy value*: We do not consider Eq. 4 to calibrate value scores with greedy results.

As shown in Table 2, we observe that dynamic budgeting helps in both *Incremental* and *Batch* by allowing them to maintain higher accuracy with fewer tokens compared to all other variants. Specifically, ⇒ *static budget* severely hurts both performance and cost, particularly leading to 3 times computation costs when using *Batch Expansion*. *w/o depth penalty* and *w/o greedy value* perform competitively for *Incremental*, but still have considerable negative influence on *Batch*. These results highlight the importance of dynamic budgeting especially in scenarios where *Batch Expansion* is employed.

Influences of Budget Limitation Budget limitation B decides the upperbound of budget size b. As illustrated in Fig. 4, we observe a clear tradeoff between performance and cost. With the growth of B, the computation cost also increases correspondingly because larger budget sizes are allocated to challenging states with lower value scores. Consequently, more problems are correctly solved due to more comprehensive searching. Regarding the two

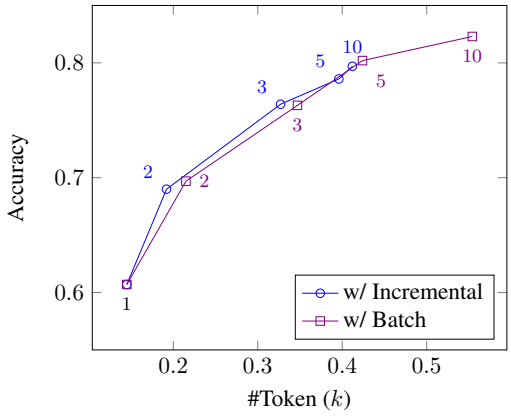

Figure 4: Performance of *Incremental* and *Batch* on GSM8K when using different budget limitations B, where $B = \{1, 2, 3, 5, 10\}$.

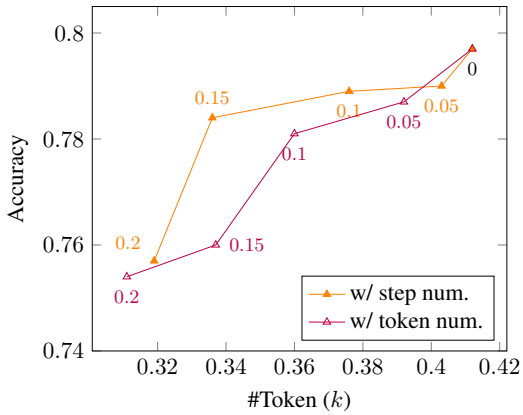

Figure 5: Performance of *Incremental Expansion* on GSM8K when using *step number* and *token number* to estimate progress term $p(\mathcal{S})$, where $\lambda = \{0, 0.05, 0.1, 0.15, 0.2\}$.

expansion strategies, *Incremental* perform slightly better than *Batch* with competitive accuracy but fewer costs when $B \leq 3$. This is because it may not use up all budgets when good nodes have been generated during incremental expansion. However, *Batch* yields better accuracy by taking more costs when $B = \{5, 10\}$ because it fully utilizes allocated budgets, thus providing larger search space for better selection.

Influence of Progress Estimation We then investigate the choice of $p(\mathcal{S})$ and λ in Eq. 2. We consider *step number* and *token number* against corresponding results of greedy decoding to estimate $p(\mathcal{S})$. As depicted in Fig. 5, increasing λ improves cost-efficiency by prioritizing nodes with faster progress at the risk of inaccuracy. Comparing *step number* and *token number*, the former is relatively better with a modest downward trend.

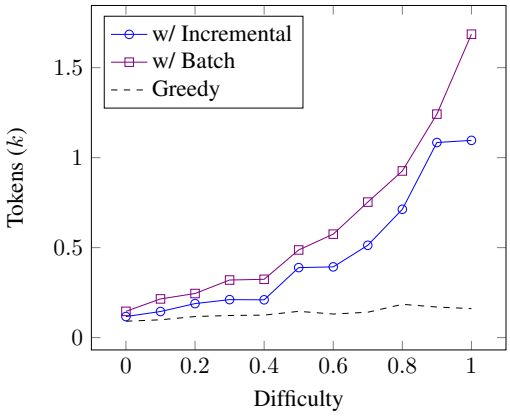

Figure 6: Cost of *Incremental* and *Batch* with the growth of difficulty on GSM8K. *Difficulty* is defined as "$1 - x$", where x is the frequency of gold answer in 20 sampled paths.

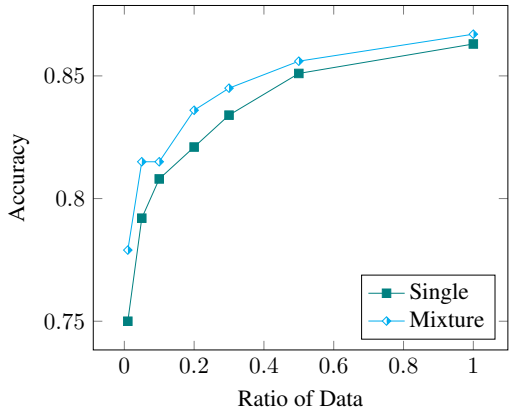

Figure 7: Accuracy of *Incremental* on TabMWP with value networks trained with *Single* or *Mixture* domains data, where we use full GSM8K and different ratio of TabMWP from 1% to 100%.

By sacrificing 1.3 points in accuracy, utilizing *step number* and $\lambda = 0.15$ saves nearly 20% computational costs. In contrast, the efficacy of *token number* is unsatisfactory. This can be attributed to its higher degree of variability, thus yielding less precise estimates of progress.

Harder Problems are Allocated Larger Budgets Fig. 6 illustrates the correlation between cost and question difficulty. Inspired by (Wang et al., 2024a), we estimate the difficulty of a question by computing the frequency of the gold answer in multiple sampled paths after inversion ("$1 - x$"). We observe that for easier questions, our methods cost competitively to *Greedy Decoding*. However, as the difficulty escalates, the cost of our method also rises proportionately. Regarding our expansion strategies, *Batch* consistently takes higher costs and the gap also widens with the difficulty increases.

Mixture-of-Domain Boosts Performance An important future direction is to construct a general value network that can address questions from different domains. To validate the potential of this direction, we conducted experiments using value networks trained with different ratios of TabMWP data and full GSM8K data. Despite the significant difference in question style and answer type, the results in Fig. 7 demonstrate that using a mixture of different domains helps improve search performance, especially when training instances from the target domain are scarce (0.75 vs. 0.78 on Accuracy when using 1% TabMWP data). This highlights the effectiveness of building robust and stronger value networks by collecting various training instances.

Question:
Josh decides to try flipping a house. He buys a house for $80,000 and then puts in $50,000 in repairs. This increased the value of the house by 150%. How much profit did he make?

Correct Solution:
Step 1. The value of the house increased by 150%, so it increased by 150% * 80,000 = 120,000. ***(Value: 0.33)***
Step 2. Thus, the value of the house is now 80,000 + 120,000 = 200,000. ***(Value: 0.49)***
Step 3. Josh put in 50,000 + 80,000 = 130,000 in the house, so he made a profit of 200,000 - 130,000 = 70,000. ***(Value: 0.97)***

Wrong Solution:
Step 1. The total cost was 80,000 + 50,000 = 130,000 ***(Value: 0.62)***
Step 2. The new value of the house was 150% of the cost or 1.5 * 130,000 = 195,000 ***(Value: 0.31)*** ✗
Step 3. So the profit was 195,000 - 130,000 = 65,000 ***(Value: 0.35)*** ✗

Figure 8: A lemon picked example. We provide correct and wrong solutions with value scores for each step and error steps are marked.

Further exploration in this direction will be pursued in future work.

4.5 Limitations of Guided Tree Search

Analyses above have shown the effectiveness of our method. However, though much more efficient, guided tree searches often cannot outperform Soft Voting on Accuracy. This section will provide detailed analyses to answer this question.

Larger Variance of Values at the First Few Steps We first collect the questions that our approach fails to solve, yet are successfully addressed by *Soft Voting@20*. Fig. 8 displays a lemon pick example. We observe that our value network can select the correct answer when the complete rationale is provided. However, the first two steps in the correct path are scored much lower than the first step of

	Accuracy ↑	Tokens (k) ↓
Best@20	.833	2.63
Soft Voting@20	.843	2.63
Ours (Batch)	.823	**0.55**
+Soft Voting ($\alpha = 0.7$)	.841	0.73
+Soft Voting ($\alpha = 0.8$)	.843	0.79
+Soft Voting ($\alpha = 0.9$)	**.847**	0.99

Table 3: Results of our methods enhanced by voting, where *Best@20* is a variant of *Soft Voting@20*, which only selects the best path with the highest value as the final output. For *Ours + Soft Voting*, we discard results with values lower than α, and utilize *Soft Voting@20* to solve them.

the wrong solution. This results in a reduced priority in exploring the node that is actually of higher quality. We also compute the average best ranking of values for correct solutions across the 20 sampled paths for these questions. Results indicate that the rank for the first step is 3.0, whereas the final step achieves a rank of 1.4. This finding highlights the larger variance of values at the first few steps, which subsequently results in inadequate exploration of high-quality nodes and finally fails to yield the correct answer.

Voting Helps when Values are Inaccurate Due to the imperfect value network, some incorrect paths may be erroneously scored higher than the correct ones. Guided tree search methods, which search for only one path as the final answer, inevitably fail in these instances. However, Soft Voting can mitigate this issue by leveraging the benefits of both majority voting and the value network. Consequently, even if the highest value is attained by an incorrect path, Soft Voting still has the potential to reach the correct answer with a higher frequency. As demonstrated in Table 3, the use of voting enables *Soft Voting@20* to outperform *Best@20*, highlighting the efficacy of voting in enhancing accuracy.

Inspired by these findings, we further investigate the improvement of our method using voting. Specifically, we discard the answers predicted by our method when their value scores fall below a threshold α. Generally, these predictions exhibit a higher error rate due to the correlation between value scores and correctness. Subsequently, we employ Soft Voting to address these unresolved questions. The results in Table 3 indicate that accuracy can be significantly improved by increasing α. However, the associated costs also rise substantially, albeit remaining lower than those of *Soft Voting@20*.

5 Conclusion

In this work, we study guided tree search to address math problems, aiming to decrease the computation costs while maintaining the performance. Inspired by the theory of value function, we propose dynamic node selection and expansion strategies, which dynamically determine the priority of nodes to explore and manage the computational budget during expansion. Both procedures are guided by an easy-to-implement value network trained without step-wise supervision. Experiments show that our methods achieve competitive performance with typical baselines but significantly save computation costs. Ablation studies validate the effectiveness of each component, providing more feasible options for various practical scenarios. Besides, we also identify the shortcomings of this research line, and provide a potential strategy for addressing these issues.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Aida Amini, Saadia Gabriel, Shanchuan Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2357–2367.

Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. 2024. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Kahneman Daniel. 2017. *Thinking, fast and slow*.

Xidong Feng, Ziyu Wan, Muning Wen, Ying Wen, Weinan Zhang, and Jun Wang. 2023. Alphazero-like tree-search can guide large language model decoding and training. *arXiv preprint arXiv:2309.17179*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

Mark Hopkins, Cristian Petrescu-Prahova, Roie Levin, Ronan Le Bras, Alvaro Herrasti, and Vidur Joshi. 2017. Beyond sentential semantic parsing: Tackling the math sat with a cascade of tree transducers. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 795–804.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088*.

Jikun Kang, Xin Zhe Li, Xi Chen, Amirreza Kazemi, and Boxing Chen. 2024. Mindstar: Enhancing math reasoning in pre-trained llms at inference time. *arXiv preprint arXiv:2405.16265*.

Muhammad Khalifa, Lajanugen Logeswaran, Moontae Lee, Honglak Lee, and Lu Wang. 2023. Grace: Discriminator-guided chain-of-thought reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 15299–15328.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213.

Jierui Li, Lei Wang, Jipeng Zhang, Yan Wang, Bing Tian Dai, and Dongxiang Zhang. 2019. Modeling intra-relation in math word problems with different functional multi-head attentions. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 6162–6167.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

Haoxiong Liu and Andrew Chi-Chih Yao. 2024. Augmenting math word problems via iterative question composing. *arXiv preprint arXiv:2401.09003*.

Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.

Pan Lu, Liang Qiu, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, Tanmay Rajpurohit, Peter Clark, and Ashwin Kalyan. 2022. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. In *The Eleventh International Conference on Learning Representations*.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*.

Qianli Ma, Haotian Zhou, Tingkai Liu, Jianbo Yuan, Pengfei Liu, Yang You, and Hongxia Yang. 2023. Let's reward step by step: Step-level reward model as the navigators for reasoning. *arXiv preprint arXiv:2310.10080*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2024. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36.

Takuya Matsuzaki, Takumi Ito, Hidenao Iwane, Hirokazu Anai, and Noriko H Arai. 2017. Semantic parsing of pre-university math problems. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2131–2141.

Jinghui Qin, Xiaodan Liang, Yining Hong, Jianheng Tang, and Liang Lin. 2021. Neural-symbolic solver for math word problems with auxiliary tasks. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5870–5881.

Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.

Mirac Suzgun and Adam Tauman Kalai. 2024. Meta-prompting: Enhancing language models with task-agnostic scaffolding. *arXiv preprint arXiv:2401.12954*.

Ye Tian, Baolin Peng, Linfeng Song, Lifeng Jin, Dian Yu, Haitao Mi, and Dong Yu. 2024. Toward self-improvement of llms via imagination, searching, and criticizing. *arXiv preprint arXiv:2404.12253*.

Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. 2024. Openmathinstruct-1: A 1.8 million math instruction tuning dataset. *arXiv preprint arXiv:2402.10176*.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Ante Wang, Linfeng Song, Ye Tian, Baolin Peng, Lifeng Jin, Haitao Mi, Jinsong Su, and Dong Yu. 2024a. Self-consistency boosts calibration for math reasoning. *arXiv preprint arXiv:2403.09849*.

Chaojie Wang, Yanchen Deng, Zhiyi Lv, Shuicheng Yan, and An Bo. 2024b. Q*: Improving multi-step reasoning for llms with deliberative planning. *arXiv preprint arXiv:2406.14283*.

Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Y Wu, and Zhifang Sui. 2023. Math-shepherd: A label-free step-by-step verifier for llms in mathematical reasoning. *arXiv preprint arXiv:2312.08935*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Qinzhuo Wu, Qi Zhang, Zhongyu Wei, and Xuan-Jing Huang. 2021. Math word problem solving with explicit numerical values. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5859–5869.

Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, James Xu Zhao, Min-Yen Kan, Junxian He, and Michael Xie. 2024. Self-evaluation guided beam search for reasoning. *Advances in Neural Information Processing Systems*, 36.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. *Preprint*, arXiv:2309.03409.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36.

Longhui Yu, Weisen Jiang, Han Shi, YU Jincheng, Zhengying Liu, Yu Zhang, James Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2023. Metamath: Bootstrap your own mathematical questions for large language models. In *The Twelfth International Conference on Learning Representations*.

Zhen Zeng, William Watson, Nicole Cho, Saba Rahimi, Shayleen Reynolds, Tucker Balch, and Manuela Veloso. 2023. Flowmind: automatic workflow generation with llms. In *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 73–81.

Di Zhang, Jiatong Li, Xiaoshui Huang, Dongzhan Zhou, Yuqiang Li, and Wanli Ouyang. 2024. Accessing gpt-4 level mathematical olympiad solutions via monte carlo tree self-refine with llama-3 8b. *arXiv preprint arXiv:2406.07394*.

Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V Le, Ed H Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. 2024. Self-discover: Large language models self-compose reasoning structures. *arXiv preprint arXiv:2402.03620*.

Tinghui Zhu, Kai Zhang, Jian Xie, and Yu Su. 2024. Deductive beam search: Decoding deducible rationale for chain-of-thought reasoning. *arXiv preprint arXiv:2401.17686*.

Below is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:
You are given a math problem, followed by a step-by-step reasoning process. Your task is to read the problem carefully, understand and follow the solving steps, then predict the chance of generating correct final answer with your internal knowledge. Output 'True' if you think the final answer will be correct, and 'False' otherwise.

Input:
Freddie and his team are collecting blankets for three days to be donated to the Children Shelter Organization. There are 15 people on the team. On the first day, each of them gave 2 blankets. On the second day, they tripled the number they collected on the first day by asking door-to-door. On the last day, they set up boxes at schools and got a total of 22 blankets. How many blankets did they collect for the three days for donation?
On the first day, they collected 15 people * 2 blankets per person = 30 blankets.
On the second day, they collected 3 times as many blankets = 90 blankets.
On the last day, they collected 22 blankets.
So in total, they collected 30 + 90 + 22 = 142 blankets.

Figure 9: An example fed to value network for scoring.